Reconocimiento de caracteres *kuzushiji* con redes convolucionales

Aprendizaje profundo con PyTorch sobre KMNIST: núcleo del taller y extras

Taller 3

Asignatura: Inteligencia Computacional Aplicada
Docente: Cesar Andrey Perdomo Charry
Estudiantes: Juan Felipe Rodríguez Galindo – 20261595004
Programa: Maestría en Ciencias de la Información y las Comunicaciones
Fecha: 10 de octubre de 2026

Universidad Distrital Francisco José de Caldas

Facultad de Ingeniería – Bogotá D.C., Colombia

Índice

1. Introducción **2**

2. Datos y protocolo experimental **2**

3. Arquitectura `CNNPropia` (TODO 2) **3**

4. Entrenamiento (TODO 3) **3**

5. Evaluación en prueba y análisis de errores (TODO 4) **4**

6. Aumento de datos (TODO 5) **5**

7. Extras **6**

7.1. Ablación de componentes y profundidad 6

7.2. Líneas base: MLP y CNN del docente 8

7.3. KMNIST frente a MNIST y FashionMNIST 8

7.4. Tasa de aprendizaje 9

7.5. Lo que aprende la red 9

8. Discusión y limitaciones **10**

9. Conclusiones **10**

1. Introducción

El taller propuesto pide repetir el flujo del cuaderno resuelto por el docente (dígitos MNIST, un perceptrón multicapa y una CNN en PyTorch) sobre **otro conjunto de datos**, completando cinco bloques de código con verificación automática y respondiendo preguntas de análisis. Se eligió **KMNIST**: 70 000 imágenes de 28×28 de hiragana cursivo (*kuzushiji*) en 10 clases, con 60 000 imágenes de entrenamiento y 10 000 de prueba.

Este informe cubre dos cosas. Primero, el **núcleo del taller** (TODO 1 a 5): conteo por clase, diseño de una red convolucional propia, bucle de entrenamiento, evaluación en prueba con análisis de errores y aumento de datos. Segundo, unos **extras** que responden preguntas que el taller deja abiertas y que se resolvieron con evidencia: varias semillas con comparaciones pareadas, ablación de componentes, comparación con las redes del docente, KMNIST frente a MNIST y FashionMNIST, barrido de la tasa de aprendizaje y visualización de lo que aprende la red.

> **Resultados principales.** `CNNPropia` (20 586 parámetros) alcanza **97,40 %** de exactitud en validación y **91,85 %** en prueba tras 5 épocas (815 errores de 10 000). El conjunto de prueba de KMNIST es más difícil que la validación (5,5 puntos de diferencia, que no aparece en MNIST ni en FashionMNIST). El aumento de datos por desplazamiento **no mejoró** la exactitud: reduce la memorización, pero no la brecha con prueba. Dropout y, sobre todo, un tercer bloque convolucional sí ayudaron (2,11 pts en prueba).

2. Datos y protocolo experimental

Partición. Del entrenamiento oficial se separan 5 000 imágenes de validación con una permutación de semilla fija (42), quedando 55 000 / 5 000 / 10 000 imágenes para entrenamiento, validación y prueba. Cada píxel se normaliza como $x' = (p/255 - \mu)/\sigma$ con $\mu = 0{,}1918$ y $\sigma = 0{,}3484$, estadísticos **solo del entrenamiento** para no filtrar información de validación ni de prueba.

Balance (TODO 1). Las clases están prácticamente balanceadas: entre 5 450 (*su*) y 5 533 (*re*) imágenes, razón máx/mín de 1,015 (figura 1). Por eso la exactitud global es una medida razonable, aunque se complementa con la matriz de confusión.

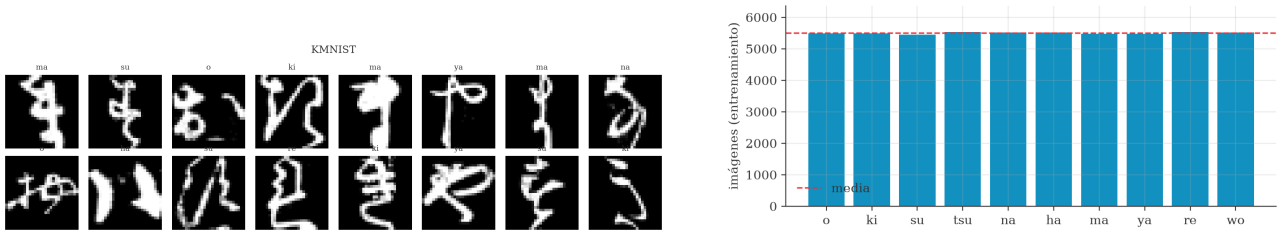


Figura 1: Muestras de KMNIST (izquierda) y conteo por clase en entrenamiento (derecha).

Reproducibilidad y comparaciones pareadas. Toda corrida fija la semilla de Python, NumPy y PyTorch. Se detectó, además, que el `DataLoader` de entrenamiento del cuaderno base comparte un único generador entre corridas: su estado avanza, así que dos entrenamientos con "la misma semilla" ven los lotes en órdenes distintos y la comparación no es realmente pareada. En las comparaciones de este informe se crea un cargador **nuevo por corrida**, de modo que, dada una semilla, la inicialización, el orden de los lotes y el *dropout* son idénticos con y sin la modificación evaluada. Los resultados se reportan como media $\pm$ desviación estándar muestral (ddof = 1) sobre 5 semillas (aumento de datos) o 3 semillas (ablación, líneas base y otros conjuntos).

3. Arquitectura `CNNPropia` (TODO 2)

La red apila dos bloques `Conv2d(3×3, relleno 1)` → `BatchNorm2d` → ReLU → `MaxPool2d(2)`, aplana, aplica `Dropout(0,3)` y termina en una capa lineal de 10 *logits*. La función de pérdida es la entropía cruzada (que incluye la *softmax*), por lo que la red no la aplica por su cuenta.

Tabla 1: Arquitectura y parámetros. Total: 20 586 ($<$ 200 000, requisito del taller).

Capa	Salida	Parámetros	Cálculo
Entrada	$1 \times 28 \times 28$	0	
Conv 3×3, 16 filtros, relleno 1	$16 \times 28 \times 28$	160	$16(1 \cdot 9 + 1)$
BatchNorm2d(16)	$16 \times 28 \times 28$	32	$2 \cdot 16$ (γ, β)
ReLU + MaxPool 2×2	$16 \times 14 \times 14$	0	
Conv 3×3, 32 filtros, relleno 1	$32 \times 14 \times 14$	4 640	$32(16 \cdot 9 + 1)$
BatchNorm2d(32)	$32 \times 14 \times 14$	64	$2 \cdot 32$
ReLU + MaxPool 2×2	$32 \times 7 \times 7$	0	
Aplanar + Dropout(0,3)	1 568	0	$32 \cdot 7 \cdot 7$
Lineal $1568 \to 10$	10	15 690	$1568 \cdot 10 + 10$

¿Por qué cada componente y qué se esperaba de él?

- **Convolución** 3×3: detecta rasgos locales (bordes, curvas, cruces de trazos) reutilizando los mismos pesos en toda la imagen; da pocos parámetros e invarianza parcial a traslaciones. Se esperaba que superara a un MLP con muchos más parámetros (confirmado en la sección 7.2). El relleno 1 conserva el tamaño antes de cada *pooling*.
- **BatchNorm**: se esperaba un entrenamiento más estable y rápido y tolerancia a tasas de aprendizaje mayores. La ablación (sección 7.1) **no** mostró mejora de exactitud a 5 épocas; el beneficio en estabilidad sí aparece en el barrido de η (sección 7.4).
- **ReLU**: no linealidad barata que no satura para valores positivos.
- **MaxPool**: divide el tamaño por 2 y da tolerancia a pequeños desplazamientos del trazo.
- **Dropout**: regulariza la capa lineal final, donde está el 76 % de los parámetros. La ablación confirma su aporte.

4. Entrenamiento (TODO 3)

Se usa Adam con $\eta = 10^{-3}$, lotes de 64 y 5 épocas. Cada iteración repite, en este orden, los cinco pasos que pide el taller: `zero_grad()` (PyTorch acumula los gradientes y sin borrarlos se mezclan lotes), propagación hacia adelante, entropía cruzada, `backward()` (regla de la cadena desde los *logits*, donde $\partial L / \partial z = p - y$) y `step()` (Adam actualiza los 20 586 parámetros a la vez).

La pérdida de entrenamiento baja de 0,374 a 0,117 y la exactitud de validación llega a 97,40 %. No hay sobreajuste visible a 5 épocas: la brecha entre exactitud de entrenamiento (en modo evaluación) y validación es de unos 0,9 puntos.

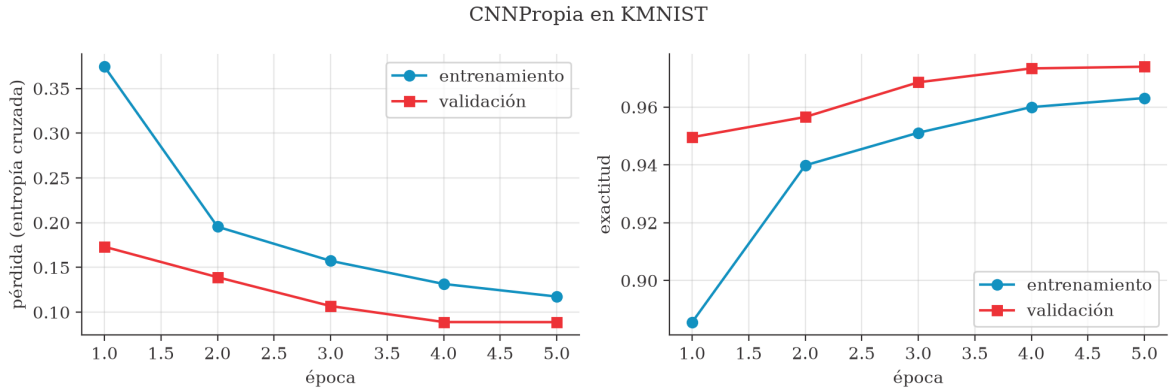


Figura 2: Pérdida y exactitud por época. La curva de entrenamiento se promedia durante la época y con *dropout* activo, por lo que queda por encima de la de validación.

Tabla 2: Historial del entrenamiento principal.

Época	Pérdida	Exact.	Pérdida val.	Exact. val.
1	0,3744	88,56 %	0,1732	94,96 %
2	0,1955	93,99 %	0,1391	95,66 %
3	0,1574	95,11 %	0,1069	96,86 %
4	0,1316	96,00 %	0,0890	97,34 %
5	0,1175	96,31 %	0,0889	97,40 %

5. Evaluación en prueba y análisis de errores (TODO 4)

El conjunto de prueba se usó para estimar el rendimiento del modelo final; ninguna decisión de diseño se tomó mirándolo. La exactitud es **91,85 %** (815 errores de 10 000).

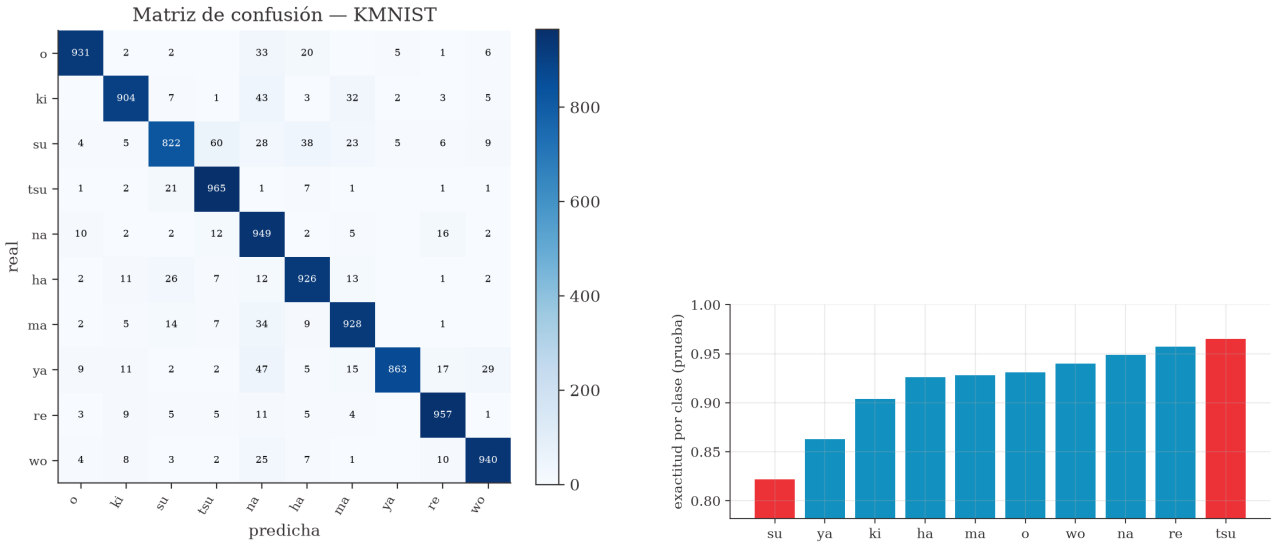


Figura 3: Matriz de confusión en prueba y exactitud por clase.

Brecha entre validación y prueba. La validación (97,40 %) supera a la prueba (91,85 %) por 5,5 puntos. No es un error del código: se repite con las 5 semillas y **no aparece** con MNIST ni FashionMNIST (sección 7.3). La validación sale de una partición aleatoria del entrenamiento y comparte su distribución; la prueba oficial de KMNIST no la comparte del todo. La causa exacta no se midió; una hipótesis razonable es que haya variantes de escritura o escritores presentes solo en prueba. Para reportar rendimiento hay que mirar la prueba; la validación sirve para *comparar* configuraciones, no para predecir el número final.

Par más confundido.

Tabla 3: Cinco pares (real → predicha) con más errores en prueba.

Real → predicha	Veces	Sentido contrario
su → tsu	60	21
ya → na	47	0
ki → na	43	2
su → ha	38	26
ma → na	34	5

El par más frecuente es **su → tsu** (60 veces; en sentido contrario, 21). *su* es además la clase más difícil (82,2 %) y *tsu* la más fácil (96,5 %). Al mirar imágenes concretas (figura 4), los *su* mal clasificados son casi siempre una **variante cursiva** distinta de la dominante en la clase (trazo vertical largo con ganchos a la derecha, sin el bucle característico de la forma estándar), parecida a varias grafías de *tsu* bien clasificadas. Es **variación dentro de la clase**: el kuzushiji conserva formas alternativas del mismo carácter. Que la confusión sea asimétrica (60 frente a 21) sugiere que *tsu* actúa como "atractor" de esas variantes raras de *su*. Algo similar ocurre con *na*, clase predicha en tres de los cinco pares más frecuentes y casi nunca confundida en sentido inverso.

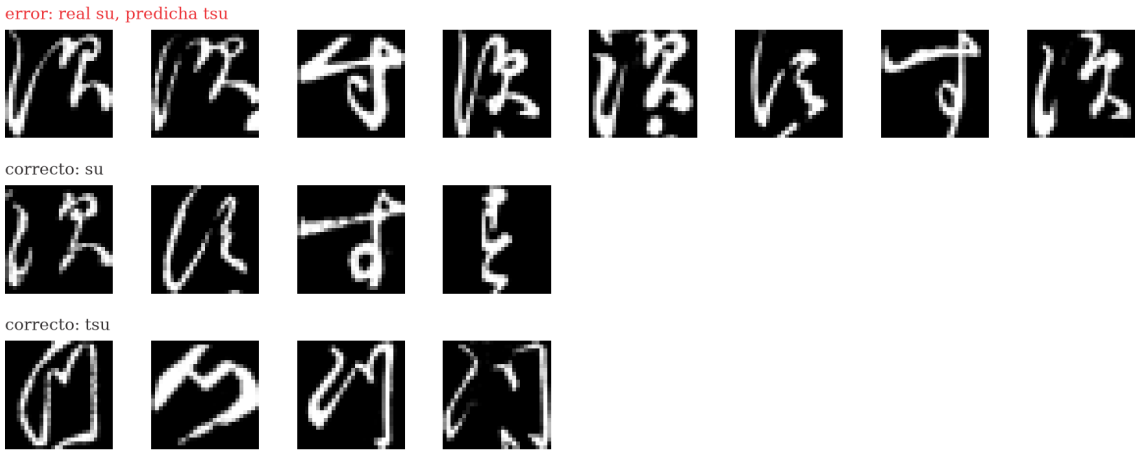


Figura 4: Arriba, errores *su* leído como *tsu*; abajo, aciertos de cada clase para contraste.

6. Aumento de datos (TODO 5)

Se implementó un desplazamiento aleatorio de hasta ±2 píxeles con `torch.roll`: uno por lote (el del taller) y, como extra, uno distinto por imagen. Se comparó con y sin aumento con 5 semillas pareadas (tabla 4).

Tabla 4: Aumento de datos, 5 épocas, 5 semillas. “Entren. (eval)” es la exactitud de entrenamiento con el modelo en modo evaluación; la brecha es entren. − validación.

Configuración	n	Entren. (eval)	Validación (%)	Prueba (%)	Brecha
sin aumento	5	98,00 %	97,10 ± 0,27	92,13 ± 0,28	0,90 pts
con aumento (lote)	5	97,30 %	97,09 ± 0,13	91,81 ± 0,15	0,21 pts
con aumento (por muestra)	5	97,43 %	97,22 ± 0,13	91,89 ± 0,24	0,22 pts

¿Mejoró la exactitud de validación? No. Frente a “sin aumento”: por lote 0,00 pts en validación y −0,33 pts en prueba (mejor en 1 de 5 semillas); por muestra 0,12 pts en validación y −0,25 pts en prueba. Las diferencias están dentro del ruido entre semillas (0,3 a 0,4 puntos).

¿Cambió la brecha entre entrenamiento y validación? Sí: baja de 0,90 puntos a unos 0,2. El aumento regulariza (la red memoriza menos), pero esa menor memorización no se traduce en mejor generalización. Posibles razones, hipótesis no probadas por separado: (a) con 5 épocas casi no hay sobreajuste que corregir; (b) convolución más *max-pooling* ya tolera desplazamientos pequeños; (c) `torch.roll` es circular y reinserta por el borde opuesto lo que sale; (d) los errores en prueba parecen venir de variantes de escritura, no de traslaciones.

¿Y con más épocas? Con 15 épocas (3 semillas) el modelo sin aumento sí empieza a memorizar y el aumento recorta esa brecha (de 1,09 pts a 0,40 pts), pero otra vez sin mejorar: −0,15 pts en validación y −0,19 pts en prueba.

Tabla 5: Aumento por muestra con 15 épocas, 3 semillas.

Configuración (15 épocas)	n	Entren. (eval)	Validación (%)	Prueba (%)	Brecha
sin aumento	3	99,32 %	98,23 ± 0,18	94,15 ± 0,18	1,09 pts
con aumento (por muestra)	3	98,48 %	98,08 ± 0,18	93,96 ± 0,14	0,40 pts

7. Extras

7.1. Ablación de componentes y profundidad

Se quita una pieza a la vez, con 3 semillas pareadas.

Tabla 6: Ablación, 5 épocas, 3 semillas.

Variante	Parám.	Entren. (eval)	Validación (%)	Prueba (%)	Brecha
completa (16-32, BN, Dropout)	20 586	98,06 %	97,15 ± 0,16	92,10 ± 0,16	0,90 pts
sin BatchNorm	20 490	98,30 %	97,47 ± 0,25	92,64 ± 0,34	0,83 pts
sin Dropout	20 586	98,14 %	96,69 ± 0,20	91,58 ± 0,35	1,45 pts
sin BatchNorm ni Dropout	20 490	98,72 %	97,26 ± 0,09	92,56 ± 0,51	1,46 pts
3 bloques (16-32-64)	29 290	99,12 %	98,23 ± 0,16	94,21 ± 0,29	0,89 pts

- **Dropout aporta**: quitarlo baja la validación −0,47 pts y la prueba −0,52 pts, y sube la brecha a 1,45 pts (frente a 0,90).
- **BatchNorm no mejoró la exactitud** a 5 épocas con $\eta = 10^{-3}$: sin ella la validación cambia 0,32 pts y la

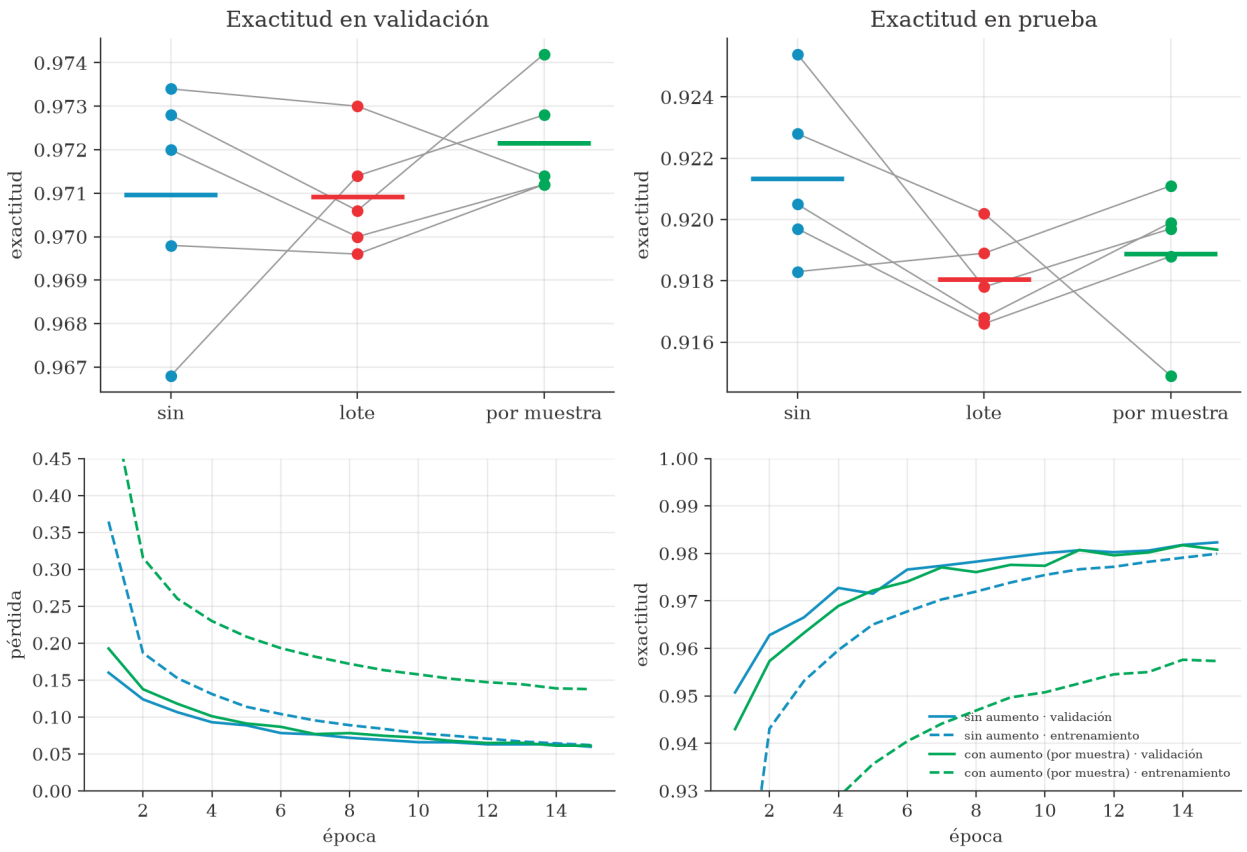
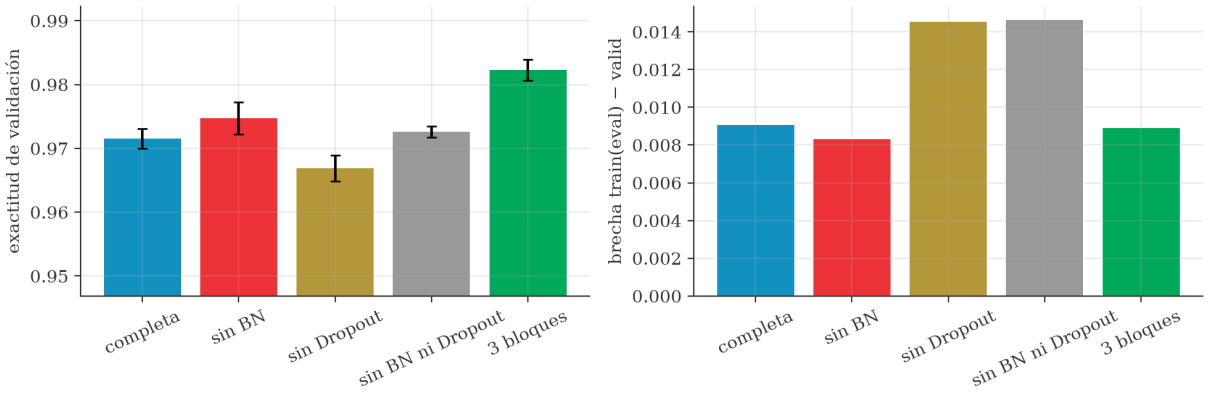


Figura 5: Arriba: una semilla por punto (las líneas grises unen la misma semilla). Abajo: curvas medias con 15 épocas.

prueba 0,54 pts (con 3 semillas es indistinguible del ruido). Esto contradice la expectativa de "estabiliza y acelera" en cuanto a la *exactitud* y se anota como tal.

- **El tercer bloque es la mejora clara**: 1,07 pts en validación y 2,11 pts en prueba con 8 704 parámetros más (+42 %). La brecha no cambia, es decir, la red base estaba **subajustada**; faltaba capacidad y campo receptivo, no regularización.



7.2. Líneas base: MLP y CNN del docente

Tabla 7: Mismas condiciones sobre KMNIST, 3 semillas. Tiempos en CPU.

Modelo	Parám.	Validación (%)	Prueba (%)	Tiempo
MLP docente (784-128-10)	101 770	95,45 ± 0,18	88,44 ± 0,59	3 s
CNN docente (12 810 par.)	12 810	96,87 ± 0,04	91,20 ± 0,37	36 s
CNNPropia (20 586 par.)	20 586	97,15 ± 0,16	92,10 ± 0,16	48 s

La CNN pequeña del docente (12 810 parámetros) supera al MLP (101 770) y generaliza mejor; `CNNPropia` gana 0,90 pts adicionales sobre esa CNN y 3,66 pts sobre el MLP (prueba: 92,10 % frente a 88,44 %). El MLP memoriza (brecha de 3,1 puntos).

7.3. KMNIST frente a MNIST y FashionMNIST

Tabla 8: La misma `CNNPropia`, sin cambios, en tres conjuntos (3 semillas).

Dataset	Validación (%)	Prueba (%)	Val.−prueba	Peor clase	Clases < 90 %
MNIST	98,72 ± 0,09	98,80 ± 0,18	−0,1 pts	97,2 %	0,0
FashionMNIST	90,34 ± 0,45	90,27 ± 0,10	0,1 pts	72,2 %	4,3
KMNIST	97,15 ± 0,16	92,10 ± 0,16	5,1 pts	86,3 %	2,0

Por exactitud en prueba, KMNIST (92,10 %) es más difícil que MNIST (98,80 %) y algo más fácil que Fashion-MNIST (90,27 %); por peor clase, KMNIST queda entre ambos. Lo distintivo de KMNIST es su **brecha validación–prueba** (5,1 pts frente a ≈ 0 en los otros dos).

Tabla 9: Barrido de η con Adam, 2 épocas, una semilla (exploratorio), con BatchNorm y sin ella (control).

η	Val. con BN	Val. sin BN	Pérdida val. (con BN)
10^{-1}	82,46 %	9,76 %	0,584
10^{-2}	96,70 %	92,62 %	0,107
10^{-3}	96,30 %	96,52 %	0,128
10^{-4}	92,38 %	—	0,265
10^{-5}	76,30 %	—	0,968

7.4. Tasa de aprendizaje

El óptimo está entre 10^{-2} (96,70 %) y 10^{-3} (96,30 %); la diferencia no es concluyente con una semilla. La CNN del docente (sin BatchNorm) se quedó en ≈ 10 % en MNIST con $\eta = 10^{-1}$, pero ese resultado mezcla red y conjunto. Por eso se hizo un **control**: la misma `CNNPropia` sin BatchNorm. Con $\eta = 10^{-1}$ cae a 9,76 % (no aprende, igual que en el script del docente) mientras que con BatchNorm llega a 82,46 %; con 10^{-2} la diferencia es 96,70 % frente a 92,62 %, y con 10^{-3} es nula (96,30 % frente a 96,52 %). Es decir, **BatchNorm no mejora la exactitud con una η bien elegida, pero ensancha el rango de η que funciona**. Es una sola semilla, pero a 10^{-1} la diferencia es de unos 73 puntos.

7.5. Lo que aprende la red

Los filtros de la primera capa se mueven (en promedio cambian 46 % de su norma respecto a la inicialización), pero con 9 pesos por filtro y 5 épocas no se ven como detectores de bordes "de libro" y no conviene forzar esa interpretación. Los mapas del bloque 1 se encienden sobre los trazos; los del bloque 2 son más dispersos y responden a fragmentos de trazo.

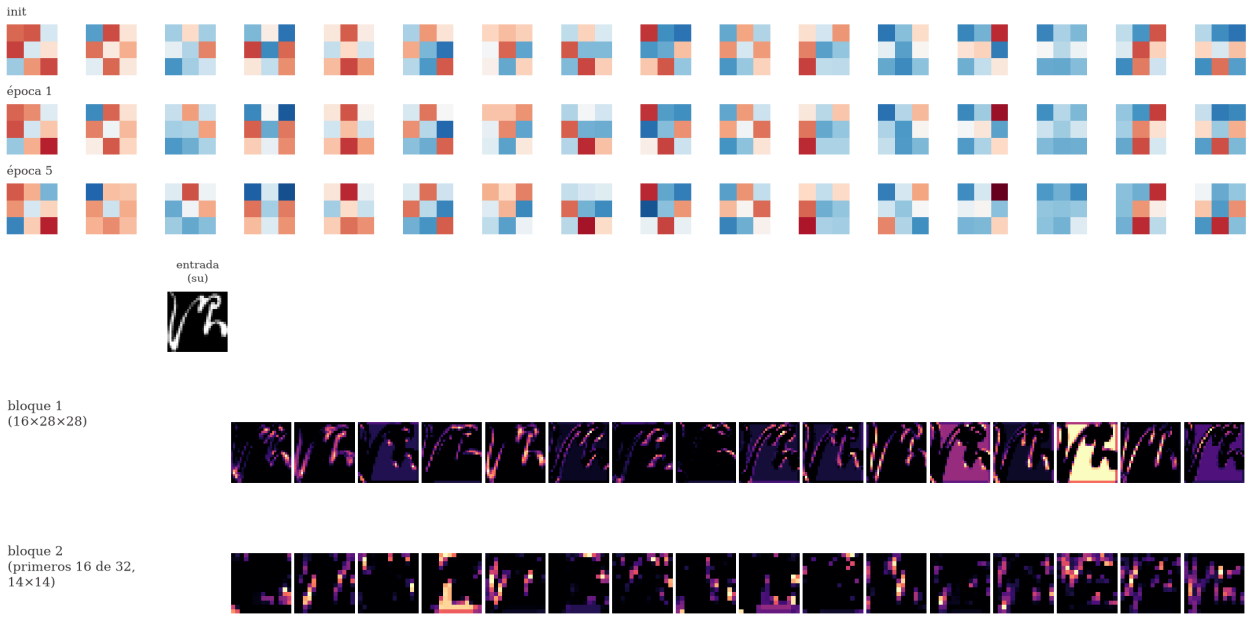


Figura 6: Filtros de la primera capa (inicio, época 1, época 5) y mapas de activación de ambos bloques para un carácter de prueba.

8. Discusión y limitaciones

- **Validación y prueba no son intercambiables en KMNIST.** Con la validación aleatoria se habría reportado ≈ 97 %; el rendimiento esperado en prueba es ≈ 92 %.
- **Lo que falla es variación intra-clase.** Los errores se concentran en variantes cursivas de un mismo carácter; más épocas o más regularización no atacan eso. Un tercer bloque (+2,1 pts) sí ayuda.
- **Un resultado nulo es un resultado.** El aumento por desplazamiento regularizó pero no mejoró; ensayar rotaciones pequeñas o deformaciones elásticas queda como trabajo futuro.
- **Alcance.** 3 a 5 semillas, 5 épocas (15 en el extra del aumento), una arquitectura base y una partición de validación fija: las diferencias de décimas de punto no son concluyentes.
- **Cómo se eligió η.** Se usó 10^{-3} por el enunciado; el barrido sugiere que 10^{-2} habría sido igual o ligeramente mejor.

9. Conclusiones

1. El taller se resolvió completo: los cinco TODO pasan sus verificaciones. `CNNPropia` (20 586 parámetros) logra 97,40 % en validación y 91,85 % en prueba.
2. En KMNIST el número que importa es el de prueba, unos 5,5 puntos por debajo del de validación; es una propiedad del conjunto, no un fallo.
3. Los errores se explican por variantes cursivas de un mismo carácter (*su*→*tsu*, y varias clases → *na*), no por desbalance.
4. El aumento por desplazamiento no mejoró la exactitud ni a 5 ni a 15 épocas, aunque redujo la memorización.
5. Dropout y más profundidad sí funcionaron; la red base estaba subajustada. Una CNN pequeña supera claramente al MLP con una fracción de los parámetros.
6. Se comparó siempre con semillas pareadas y se reportaron los resultados negativos tal cual.

Reproducibilidad

Todo el código está en `desarrollo/` (`build_notebook.py` genera el cuaderno `Taller_3_CNN_KMNIST.ipynb`); `make notebook` lo ejecuta completo (aprox. 1 h en CPU; las corridas costosas se guardan en `resultados/corridas.jsonl`) y `make rapido` ejecuta solo el núcleo. Versiones: Python 3.12, PyTorch 2.14.1, torchvision 0.29.1. Semillas de las comparaciones: 0–4 (aumento) y 0–2 (ablación, bases, conjuntos); partición con semilla 42. La animación interactiva está en `animacion/index.html`.

Referencias

[1] T. Clanuwat *et al.*, “Deep Learning for Classical Japanese Literature,” arXiv:1812.01718, 2018.

[2] Y. LeCun, L. Bottou, Y. Bengio y P. Haffner, “Gradient-based learning applied to document recognition,” *Proc. IEEE*, 86(11), 1998.

[3] H. Xiao, K. Rasul y R. Vollgraf, “Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms,” arXiv:1708.07747, 2017.

[4] S. Ioffe y C. Szegedy, “Batch Normalization: accelerating deep network training by reducing internal covariate shift,” *ICML*, 2015.

[5] N. Srivastava *et al.*, “Dropout: a simple way to prevent neural networks from overfitting,” *JMLR*, 15, 2014.

[6] D. P. Kingma y J. Ba, “Adam: a method for stochastic optimization,” *ICLR*, 2015.

[7] A. Paszke *et al.*, “PyTorch: an imperative style, high-performance deep learning library,” *NeurIPS*, 2019.